

Reporte técnico del proyecto APEGHaskell

Curso de Compiladores

Julio de 2026

Cuadro 1: Integrantes del proyecto

Integrante	Participacion
Yoselyn Victoria Miranda Chirinos	100 %
Rolando David Diaz Jara	100 %

Índice

1. Introducción	3
2. Objetivos del proyecto	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Tecnologías utilizadas	3
3.1. Herramientas principales	4
3.2. Dependencias principales	4
4. Estructura general del proyecto	4
5. Representación abstracta de gramáticas	5
5.1. Gramática	5
5.2. Regla APEG	6
5.3. Expresiones APEG	6
5.4. Tipos	7
6. DSL para construir gramáticas	7
7. Manejo del estado del parser	8
7.1. Estado puro	8
7.2. Mónada de estado	9
8. Valores del intérprete	9

9. Combinadores operacionales	10
10. Intérprete APEG	10
10.1. Función <code>interp</code>	10
10.2. Evaluación de expresiones	11
10.3. Ejecución de gramática	11
11. Sistema de tipos	12
12. Árbol de derivación	12
13. Gramática Sugar	13
14. Extensiones de Sugar	14
14.1. <code>repeat...until</code>	14
14.2. <code>unless</code>	14
14.3. Composición con <code>joinRules</code>	14
15. Flujo de ejecución	15
15.1. Ejemplo con <code>x := 5;</code>	15
16. Ejecución del proyecto	15
17. Resultado del parser	16
18. Ejemplos	16
19. Limitaciones de diseño	29
20. Conclusiones	29

1. Introducción

El proyecto **APEGHaskell** es un prototipo desarrollado en Haskell para representar, verificar e interpretar **APEGs** (*Adaptable Parsing Expression Grammars*).

Una APEG es una variante de las PEG tradicionales (*Parsing Expression Grammars*) donde la gramática puede ser tratada como un dato manipulable. Esto permite que un lenguaje pueda extender su propia sintaxis durante el proceso de parsing.

El objetivo principal del proyecto es demostrar cómo una gramática puede:

- representarse como una estructura de datos,
- ser verificada mediante un sistema de tipos,
- ejecutarse mediante un intérprete,
- y extenderse con nuevas reglas sin modificar directamente el motor del parser.

El ejemplo principal del proyecto es **Sugar**, un pequeño lenguaje imperativo con soporte para extensiones sintácticas.

2. Objetivos del proyecto

2.1. Objetivo general

Implementar un prototipo de intérprete para gramáticas APEG en Haskell, capaz de reconocer entradas de texto y permitir extensiones dinámicas de la gramática.

2.2. Objetivos específicos

- Definir una representación abstracta para gramáticas APEG.
- Implementar un DSL en Haskell para construir gramáticas de forma más legible.
- Implementar un sistema de tipos para validar reglas y gramáticas.
- Implementar un intérprete para ejecutar gramáticas sobre entradas de texto.
- Generar árboles de derivación como resultado del parsing.
- Demostrar extensibilidad mediante el lenguaje Sugar.
- Agregar extensiones como `repeat...until` y `unless` usando composición de reglas.

3. Tecnologías utilizadas

El proyecto está implementado en **Haskell**.

3.1. Herramientas principales

Herramienta	Uso
Haskell	Lenguaje principal del proyecto
GHC 9.6.7	Compilador configurado en <code>stack.yaml</code>
Stack	Construcción y ejecución del proyecto
Cabal / hpack	Configuración del paquete mediante <code>package.yaml</code>

3.2. Dependencias principales

El archivo `package.yaml` define las dependencias:

```
base
containers
mtl
QuickCheck
parsec
```

Uso general de las dependencias:

- `base`: librería estándar de Haskell.
- `containers`: estructuras como `Map`.
- `mtl`: soporte para mónadas como `State`.
- `parsec`: construcción de parsers experimentales.
- `QuickCheck`: dependencia para pruebas, aunque la suite de tests aún no está desarrollada.

4. Estructura general del proyecto

La estructura principal del proyecto es:

```
app/
  Main.hs

src/APEG/
  AbstractSyntax.hs
  DSL.hs
  Combinators.hs
  TypeSystem.hs
  Playground.hs

src/APEG/Interpreter/
  APEGInterp.hs
  State.hs
  MonadicState.hs
```

```

Value.hs
DT.hs
MaybeString.hs
TypeEnvironment.hs

src/APEG/ASTSamples/
  MicroSugar.hs
  Extensions.hs
  Simple.hs
  DataDependentLanguage.hs
  DataDependentLanguageMeta.hs

src/APEG/Parser/
  APEGParser.hs
  APEGExprParser.hs

inputSamples/
  *.txt

```

El código está dividido en varias responsabilidades:

Carpeta / archivo	Responsabilidad
app/Main.hs	Ejecutable principal
src/APEG/AbstractSyntax.hs	Definición de la sintaxis abstracta
src/APEG/DSL.hs	Helpers para construir gramáticas
src/APEG/Combinators.hs	Reglas operacionales del parser
src/APEG/TypeSystem.hs	Sistema de tipos
src/APEG/PlayGround.hs	Funciones para ejecutar gramáticas
src/APEG/Interpreter/	Núcleo del intérprete
src/APEG/ASTSamples/	Gramáticas de ejemplo
inputSamples/	Archivos de entrada para pruebas

5. Representación abstracta de gramáticas

Archivo principal:

```
src/APEG/AbstractSyntax.hs
```

Este archivo define las estructuras centrales del proyecto.

5.1. Gramática

Una gramática APEG se representa como una lista de reglas:

```
type ApegGrm = [ApegRule]
```

Esto es importante porque convierte la gramática en un valor manipulable dentro del programa.

5.2. Regla APEG

Una regla se define como:

```
data ApegRule = ApegRule NonTerminal [(Type, Var)] [(Type, Expr)] APeg
```

Cada regla tiene:

1. un nombre de no terminal,
2. una lista de atributos heredados,
3. una lista de atributos sintetizados,
4. un cuerpo de tipo `APeg`.

5.3. Expresiones APEG

El cuerpo de una regla se representa con:

```
data APeg
  = Lambda
  | Lit String
  | NT NonTerminal [Expr] [Var]
  | Kle APeg
  | Not APeg
  | Seq APeg APeg
  | Alt APeg APeg
  | Update [(Var, Expr)]
  | Constr Expr APeg
  | Bind Var APeg
  | Ann Annot APeg
```

Significado de los constructores principales:

Constructor	Significado
<code>Lambda</code>	expresión vacía
<code>Lit</code>	literal de texto
<code>NT</code>	llamada a un no terminal
<code>Kle</code>	repetición *
<code>Not</code>	predicado negativo
<code>Seq</code>	secuencia
<code>Alt</code>	alternativa ordenada
<code>Update</code>	actualización de atributo
<code>Constr</code>	restricción booleana
<code>Bind</code>	captura del texto consumido

Constructor	Significado
Ann	anotación para modificar el árbol de derivación

5.4. Tipos

También se define un sistema de tipos propio:

```
data Type
  = TyStr
  | TyBool
  | TyInt
  | TyAPeg
  | TyGrammar
  | TyMap Type
  | TyRule [Type] [Type]
  | TyMetaAPeg
  | TyMetaExp
  | TyMetaType
  | TyLanguage
```

Estos tipos permiten validar atributos, reglas, expresiones y gramáticas completas.

6. DSL para construir gramáticas

Archivo:

```
src/APEG/DSL.hs
```

Este módulo define funciones auxiliares para construir gramáticas de forma más simple.

Ejemplos:

```
lit "if"
seqs [p1, p2, p3]
alts [p1, p2]
star p
star1 p
call "expr" [g] []
```

También define operadores:

Operador	Significado
(.=.)	bind/captura
(.<.)	update/asignación de atributo
(./.)	alternativa
(.:.)	secuencia
‘(	+

Operador	Significado
)‘	suma
‘(	-
)‘	resta
‘(	<
)‘	menor que
‘(	=
)‘	igualdad
(<+:)	unión de gramáticas/lenguajes
‘(	?
)‘	restricción

Este módulo es usado principalmente por las gramáticas de ejemplo, como `MicroSugar.hs` y `Extensions.hs`.

7. Manejo del estado del parser

El intérprete necesita mantener información durante el parsing. Esa información se almacena en un estado.

7.1. Estado puro

Archivo:

```
src/APEG/Interpreter/State.hs
```

Tipo principal:

```
newtype PureState = PureState
  ( VEnv
  , TyEnv
  , MybStr
  , Input
  , Result
  )
```

El estado guarda:

Campo	Descripción
VEnv	entorno de valores
TyEnv	entorno de tipos
MybStr	texto consumido desde el último bind
Input	entrada restante
Result	resultado actual del parser

7.2. Mónada de estado

Archivo:

```
src/APEG/Interpreter/MonadicState.hs
```

El estado se maneja mediante la mónada `State`:

```
type APegSt = State PureState
```

Esto permite que las operaciones del parser lean y modifiquen el estado sin tener que pasarlo manualmente como argumento.

Ejemplos de operaciones:

```
isOk  
pfail  
done  
patternMatch  
var  
varSet  
getTyEnv
```

En términos simples, `APegSt` permite que el parser tenga una memoria compartida que se va actualizando paso a paso.

8. Valores del intérprete

Archivo:

```
src/APEG/Interpreter/Value.hs
```

El tipo `Value` representa los valores que pueden existir durante la interpretación:

```
data Value  
  = VStr String  
  | VInt Int  
  | VFloat Float  
  | VBool Bool  
  | VMap (M.Map String Value)  
  | VLn ApegGrm TyEnv  
  | VGrm ApegGrm  
  | VPeg APeg  
  | VExp Expr  
  | VType Type  
  | Undefined
```

Un aspecto importante es que una gramática puede ser un valor:

```
VGrm ApegGrm  
VLn ApegGrm TyEnv
```

Esto permite que las gramáticas puedan componerse y extenderse durante el parsing.

9. Combinadores operacionales

Archivo:

```
src/APEG/Combinators.hs
```

Este módulo implementa las operaciones básicas del formalismo PEG/APEG.

Funciones principales:

Función	Operación
<code>sequential</code>	secuencia
<code>alternate</code>	alternativa ordenada
<code>klenne</code>	repetición *
<code>notPeg</code>	predicado negativo
<code>bindApeg</code>	captura de texto
<code>update</code>	actualización de variable
<code>constraintApeg</code>	restricción booleana

Ejemplo de alternativa:

```
alternate l r = do
  s <- get
  l
  onSuccess (return ()) (put s >> r)
```

Esta implementación muestra el comportamiento de PEG: si la alternativa izquierda falla, se restaura el estado y se prueba la alternativa derecha.

10. Intérprete APEG

Archivo principal:

```
src/APEG/Interpreter/APEGInterp.hs
```

Este módulo ejecuta las gramáticas.

10.1. Función `interp`

La función central es:

```
interp :: APeg -> APegSt ()
```

Esta función recibe una expresión APEG y ejecuta la operación correspondiente.

Ejemplos:

```
interp Lambda      = done
interp (Lit s)     = patternMatch s
interp (Kle p)     = klenne (interp p)
interp (Seq e d)   = sequential (interp e) (interp d)
interp (Alt e d)   = alternate (interp e) (interp d)
interp (Not e)     = notPeg (interp e)
interp (Bind v p)  = bindApeg v (interp p)
```

Esto conecta la representación abstracta (APeg) con la ejecución real.

10.2. Evaluación de expresiones

La función:

```
evalExp :: Expr -> APegSt Value
```

evalúa expresiones de atributos.

Puede evaluar:

- literales enteros,
- strings,
- variables,
- operaciones binarias,
- mapas,
- accesos a mapas,
- gramáticas,
- reglas creadas dinámicamente,
- expresiones meta.

10.3. Ejecución de gramática

La función principal es:

```
interpGrammar :: [(Var,Value)] -> ApegGrm -> APegSt ()
```

Su flujo general es:

1. verifica la gramática con `typeGrammar`,
2. si no hay errores, llama a `execGrm`,
3. `execGrm` coloca la gramática activa en el entorno como variable `g`,

4. se ejecuta la primera regla de la gramática.

La variable `g` es esencial, porque representa la gramática activa durante la ejecución.

11. Sistema de tipos

Archivo:

```
src/APEG/TypeSystem.hs
```

El sistema de tipos valida reglas, expresiones y gramáticas antes de ejecutarlas.

Funciones relevantes:

Función	Rol
<code>inferTypeExpr</code>	infiere el tipo de una expresión
<code>inferTypeMpeg</code>	infiere el tipo de un meta-PEG
<code>inferTypeMExpr</code>	infiere el tipo de una meta-expresión
<code>inferPegType</code>	verifica una expresión APEG
<code>inferTypeRule</code>	verifica una regla
<code>typeGrammar</code>	verifica una gramática completa

El intérprete llama a `typeGrammar` antes de ejecutar una gramática. Esto evita ejecutar gramáticas mal formadas o con errores de tipos.

12. Árbol de derivación

Archivo:

```
src/APEG/Interpreter/DT.hs
```

El parser produce un árbol de derivación.

Tipo principal:

```
data DT a
  = DTN String a [DT a]
  | DTT a String
  | DTNIL
```

Donde:

- `DTN` representa un nodo de no terminal,
- `DTT` representa una hoja terminal,
- `DTNIL` representa un nodo vacío.

El árbol se imprime cuando la entrada es aceptada.

Funciones relevantes:

```
dtRoot  
dtLeaf  
dtFlat  
pprintDT
```

13. Gramática Sugar

Archivo:

```
src/APEG/ASTSamples/MicroSugar.hs
```

Sugar es el lenguaje principal de ejemplo.

Incluye construcciones como:

Construcción	Sintaxis
Bloque	{ stmt+ }
Asignación	x := expr;
Lectura	read(x);
Impresión	print(expr);
Condicional	if(expr) block
Bucle	loop(expr) block
Definición de extensión	define nombre { reglas }
Activación de extensión	syntax nombre { ... }

La gramática está definida como:

```
microSugar :: ApegGrm
```

Algunas reglas principales son:

Regla	Función
ruleProg	programa completo
ruleNewSyn	parsea define
ruleRule	parsea reglas dentro de define
ruleExtStmt	parsea syntax
ruleBlock	parsea bloques
ruleStmt	parsea statements
ruleExpr	parsea expresiones
ruleCExpr	parsea comparaciones
ruleFator	parsea factores

14. Extensiones de Sugar

Archivo:

```
src/APEG/ASTSamples/Extensions.hs
```

Este módulo extiende Sugar con dos construcciones nuevas:

14.1. `repeat...until`

Sintaxis:

```
repeat {  
  ...  
} until (condicion);
```

Regla:

```
ruleRepeatUntil
```

14.2. `unless`

Sintaxis:

```
unless (condicion) {  
  ...  
}
```

Regla:

```
ruleUnless
```

14.3. Composición con `joinRules`

La gramática extendida se construye así:

```
microSugarExt = joinRules microSugar [ruleRepeatUntil, ruleUnless]
```

`joinRules` agrega las reglas nuevas como alternativas de `stmt`, siempre que tengan el mismo nombre y la misma firma que la regla original.

Esto demuestra una propiedad clave del proyecto: se puede extender el lenguaje sin modificar el intérprete.

15. Flujo de ejecución

El flujo general para ejecutar una entrada es:

```
runGrammar
  interpGrammar
  typeGrammar
  execGrm
  interp
  combinadores
  resultado
```

15.1. Ejemplo con { x := 5; }

Entrada:

```
{ x := 5; }
```

Flujo conceptual:

```
runGrammar
  interpGrammar
  typeGrammar
  execGrm
  prog
  block
  stmt
  assignment
  identifier consume x
  lit "!=" consume :=
  expr cexpr fator num consume 5
  semic consume ;
  block consume }
  ACCEPTED
```

El parser baja desde la regla inicial `prog` hasta reconocer un bloque, luego un statement, y finalmente una asignación.

Importante: Suger en este prototipo funciona principalmente como reconocedor sintáctico. Reconoce que `x := 5;` tiene forma de asignación, pero no implementa una semántica completa de ejecución de variables del lenguaje objeto.

16. Ejecución del proyecto

Desde la raíz del proyecto, se puede compilar con:

```
stack build
```

Para ejecutar un archivo de ejemplo mediante el ejecutable:

```
stack exec APEGLittleToy-exe -- inputSamples/MicrSugarForEach.txt
```

Para probar las extensiones `repeat` y `unless`:

```
stack exec APEGLittleToy-exe -- inputSamples/testExtension.txt
```

También puede ejecutarse desde GHCi importando los módulos de ejemplo.

Ejemplo para Sugar:

```
printf 'import APEG.ASTSamples.MicroSugar\nrunMS "inputSamples/Sample4.txt"\n'\n| stack ghci
```

Ejemplo para extensiones:

```
printf 'import APEG.ASTSamples.Extensions\nrunExt "inputSamples/testExtension.\ntxt"\n' | stack ghci
```

17. Resultado del parser

El resultado del parser se muestra en la terminal.

Si la entrada es aceptada, se imprime un árbol de derivación y el mensaje:

```
ACCEPTED
```

Si la entrada es rechazada, se imprime:

```
REJECTED
```

junto con errores y parte del input restante.

La impresión del resultado se realiza principalmente desde:

```
src/APEG/PlayGround.hs
```

mediante la función:

```
runGrammar
```

18. Ejemplos

Sample 2:

```
define sfor {  
    stmt -> whites "for" whites expr whites "to" whites expr whites block;  
}  
syntax sfor {
```



```
j := 0;
for j+1 to j+10 {
    print(j);
}
{
    i := 0;
    x := i+1;
    if (i < 10) {
        i := i + 1;
    }
}
```

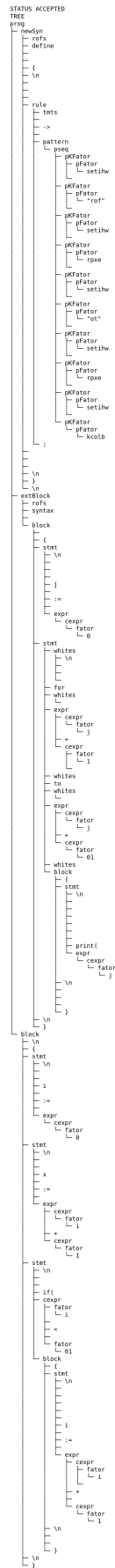


Figura 1: Resultado de la ejecución del sample 2

MicrSugarSQL:

```
define query {
  stmt      -> whites identifier whites "<--?" whites SQLExp whites ";";
  SQLExp    -> "SELECT" whites1 fieldList whites1
              "FROM" whites1 identifier
              (whites "," whites1 identifier)*
              ((whites1 "WHERE" whites1 condition) / $);
  fieldList -> field (whites "," whites field)* / "*";
  field     -> identifier (("." identifier) / $);
  condition -> SQLCond
              (whites1 "AND" whites1 SQLCond)*;
  SQLCond   -> field whites OP whites identifier;
  OP        -> "==" / "!=" / "<" / ">";
}

syntax query {
  x <--? SELECT * FROM table;
  y <--? SELECT phonebook.addres FROM phonebook
        WHERE phonebook.lastname == connor;
  z := x + 10;
}
```

Phylogenetic tree of the 20S proteasome subunit (P20) from various species. The tree is rooted at the bottom with Homo sapiens (Hs) and branches upwards. The tree is divided into several major clades, including Hs, Rn, Gg, and others. The tree is labeled with accession numbers and species names.

20

Define sfor:

```
define sfor {  
    stmt -> whites "for" whites expr whites "to" whites expr whites block;  
}  
  
syntax sfor {  
    j := 0;  
    for j+1 to j+10 {  
        print(j);  
    }  
}
```

```

STATUS ACCEPTED
TREE
prog
├── newSyn
├── rofs
├── define
├── {
├── \n
├── rule
├── tmts
├── ->
├── pattern
├── pseq
├── pKFator
├── pFator
├── setihw
├── pKFator
├── pFator
├── "rof"
├── pKFator
├── pFator
├── setihw
├── pKFator
├── pFator
├── rpxe
├── pKFator
├── pFator
├── setihw
├── pKFator
├── pFator
├── "ot"
├── pKFator
├── pFator
├── setihw
├── pKFator
├── pFator
├── rpxe
├── pKFator
├── pFator
├── setihw
├── pKFator
├── pFator
├── kcolb
├── ;
├── \n
├── }
├── \n
├── \n
├── extBlock
├── rofs
├── syntax
├── block
├── {
├── stmt
├── \n
├── j
├── :=
├── expr
├── cexpr
├── fator
├── 0
├── stmt
├── whites
├── \n
├── for
├── whites
├── expr
├── cexpr
├── fator
├── j
├── +
├── cexpr
├── fator
├── 1
├── whites
├── to
├── whites
├── expr
├── cexpr
├── fator
├── j
├── +
├── cexpr
├── fator
├── 01
├── whites
├── block
├── {
├── stmt
├── \n
├── print(
├── expr
├── cexpr
├── fator
├── j
├── \n
├── }
├── \n
├── }

```

Figura 3: Resultado de la ejecución del ejemplo

Define foreach:

```
define foreach {  
  stmt -> whites "foreach(" whites identifier whites  
    ":" whites expr whites ")" block;  
}  
  
syntax foreach {  
  x := 10;  
  foreach (i : x) {  
    y := i + 10;  
  }  
}
```

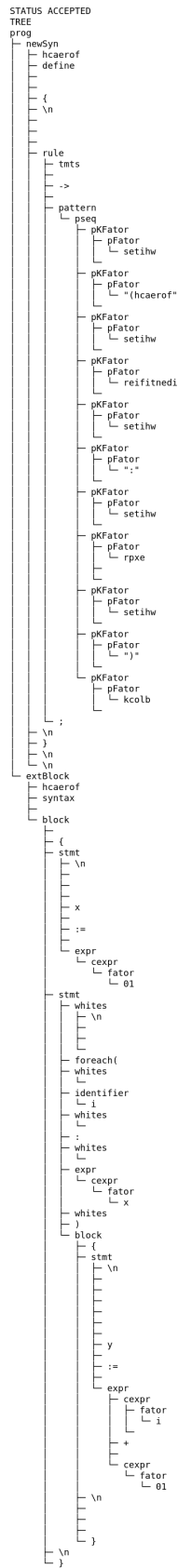


Figura 4: Resultado de la ejecución del ejemplo

Pair:


```

define pair {
  fator -> "(" whites expr whites "," whites expr whites ")";
}

syntax pair {
  x := (2,1) + (1,2);
}

```

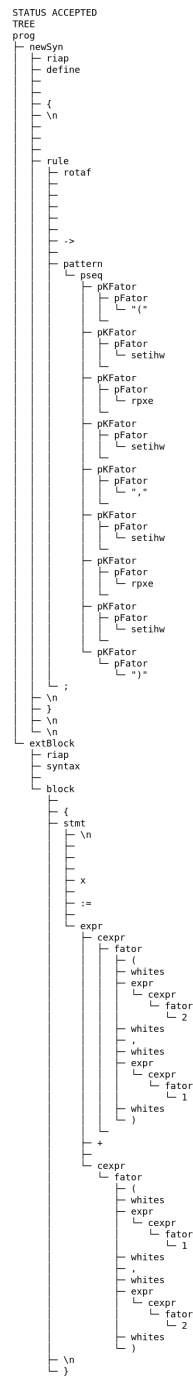


Figura 5: Resultado de la ejecución del ejemplo

Query y foreach:

```
define foreach {
  stmt -> whites "for(" whites identifier whites
        ":" whites expr whites ")" block;
}

define query {
  stmt      -> whites identifier whites "<--?" whites
              SQLexp whites ";";
  SQLexp    -> "SELECT" whites1 fieldList whites1
              "FROM" whites1 identifier
              (whites "," whites1 identifier)*
              ((whites1 "WHERE" whites1 condition) / $);
  fieldList -> field (whites "," whites field)* / "*";
  field     -> identifier (("." identifier) / $);
  condition -> SQLCond
              (whites1 "AND" whites1 SQLCond)*;
  SQLCond   -> field whites OP whites identifier;
  OP        -> "==" / "!=" / "<" / ">";
}

syntax query, foreach {
  x <--? SELECT * FROM table;
  for(z : x) {
    print(z);
  }
}
```

Figure 2 displays 12 phylogenetic trees, each corresponding to a different species. The trees are arranged vertically and are labeled with the species name and a scale bar. The species names are: 1. Arabidopsis thaliana, 2. Arabidopsis lyrata, 3. Arabidopsis halleri, 4. Arabidopsis lyrata, 5. Arabidopsis thaliana, 6. Arabidopsis lyrata, 7. Arabidopsis halleri, 8. Arabidopsis lyrata, 9. Arabidopsis thaliana, 10. Arabidopsis lyrata, 11. Arabidopsis halleri, 12. Arabidopsis lyrata. Each tree shows the evolutionary relationships between the species, with branches labeled with bootstrap values. The scale bars represent genetic distance in substitutions per site.

Extensiones repeat / unless:

```
{
  x := 5;
  repeat {
    print(x);
    x := x - 1;
  } until (x = 0);

  unless (x = 1) {
    print(x);
  }
}
```

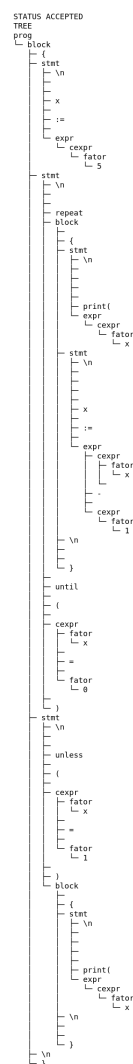


Figura 7: Resultado de la ejecución del ejemplo

19. Limitaciones de diseño

Durante el desarrollo del proyecto se identificaron tres limitaciones principales relacionadas con el diseño de las gramáticas PEG y su mecanismo de extensibilidad.

1. **Distinción entre palabras reservadas e identificadores.** Una gramática puede aceptar una palabra reservada, como `if`, como si fuera un identificador válido. Para evitar esta ambigüedad, es necesario utilizar predicados de negación que comprueben explícitamente que una entrada no corresponde a una palabra reservada antes de reconocerla como identificador. Sin embargo, esta restricción debe definirse manualmente en la gramática.
2. **Manejo de la precedencia de operadores.** PEG no proporciona un mecanismo nativo para establecer la precedencia entre operadores. Por ello, esta debe representarse mediante una jerarquía explícita de reglas, donde, por ejemplo, `expr` llama a `cexpr` y esta, a su vez, llama a `fator`. Esta estructura es rígida, ya que incorporar un nuevo operador con un nivel de precedencia diferente puede requerir la modificación y reorganización de las reglas existentes.
3. **Dependencia del orden de las alternativas.** Las alternativas en PEG se evalúan de manera ordenada y el parser selecciona la primera que produce una coincidencia. Como consecuencia, al incorporar una nueva regla a una alternativa existente, una regla anterior demasiado general puede reconocer la entrada antes de que la nueva extensión sea evaluada. Esto puede provocar que una extensión quede inaccesible durante el proceso de parsing.

20. Conclusiones

El proyecto APEGHaskell implementa una arquitectura clara para representar e interpretar gramáticas adaptables en Haskell.

Su principal aporte técnico es tratar las gramáticas como valores de primera clase. Esto permite que una gramática pueda ser combinada, extendida y pasada como parte del estado del parser.

La arquitectura se divide en:

- representación abstracta (`AbstractSyntax.hs`),
- construcción de gramáticas (`DSL.hs`),
- estado del intérprete (`State.hs`, `MonadicState.hs`),
- evaluación e interpretación (`APEGInterp.hs`),
- reglas operacionales (`Combinators.hs`),
- sistema de tipos (`TypeSystem.hs`),
- ejemplos y extensiones (`MicroSugar.hs`, `Extensions.hs`).

La extensión de Sugar con `repeat...until` y `unless` demuestra que es posible agregar nuevas construcciones sintácticas sin modificar el núcleo del intérprete. Esto se logra mediante `joinRules`, que compone reglas nuevas con reglas existentes.

En resumen, el proyecto muestra cómo Haskell puede usarse para construir un intérprete modular de gramáticas extensibles, manteniendo separadas la representación del lenguaje, el sistema de tipos, el estado de ejecución y las extensiones sintácticas.